

bigIRT Laplace-EM Algorithm

This note describes the current `marginalApprox = "laplace_em"` fitting path as it is implemented in `bigIRT`. It is intentionally implementation-facing: the goal is to document the objective actually being optimized, the approximations used, and the points where the current algorithm is only a generalized-EM approximation rather than exact marginal maximum likelihood.

Model

For observation r , let:

- $j(r)$ be the person index
- $i(r)$ be the item index
- $y_r \in \{0, 1\}$ be the response
- $\theta_{j(r)} \in \mathbb{R}^K$ be the latent ability vector
- $a_r \in \mathbb{R}^K$ be the effective row loading vector
- b_r be the effective row difficulty/intercept
- c_r be the effective lower asymptote
- d_r be the effective upper asymptote

The rowwise linear predictor is

$$\eta_r = a_r^\top \theta_{j(r)} - b_r.$$

The response model is the rowwise 4PL form

$$p_r = c_r + (d_r - c_r) \sigma(\eta_r), \quad \sigma(x) = \frac{1}{1 + e^{-x}}.$$

The 2PL and 3PL cases are obtained by fixing c_r and/or d_r .

Parameterization used in code

The optimizer works with unconstrained raw parameters and maps them into the admissible IRT scale:

$$a = \log(1 + e^\alpha), \quad c = \frac{1}{2}\sigma(\gamma), \quad d = \frac{1}{2}\sigma(\delta) + \frac{1}{2}.$$

So:

- discrimination parameters are nonnegative
- guessing parameters lie in $(0, 0.5)$
- upper asymptotes lie in $(0.5, 1)$

Item and person covariates are applied before these transforms. In the current implementation, the Laplace backend first constructs row-effective quantities:

$$a_r, b_r, c_r, d_r, \theta_{j(r)} + \text{offset}_r$$

and then passes those rowwise values into the C++ backend.

Person-step objective

For person j , conditional on the current item/global parameters, the algorithm maximizes the conditional log posterior

$$\ell_j(\theta_j) = -\frac{1}{2}(\theta_j - \mu_j)^\top Q_{0j}(\theta_j - \mu_j) + \sum_{r \in \mathcal{R}_j} \log p(y_r \mid \theta_j),$$

where:

- $\mathcal{R}_j$ is the set of rows for person j
- μ_j is the working prior mean
- Q_{0j} is the working prior precision

In the current implementation, Q_{0j} is fixed during the Laplace loop and is derived from the initialization fit after rescaling the latent space.

Row derivatives

Define:

$$g_r = \sigma(\eta_r), \quad q_r = g_r(1 - g_r), \quad u_r = d_r - c_r,$$

$$p_r = c_r + u_r g_r, \quad s_r = u_r q_r.$$

Then the row log-likelihood derivative with respect to η_r is

$$\frac{\partial \log p(y_r \mid \theta)}{\partial \eta_r} = \frac{y_r - p_r}{p_r(1 - p_r)} s_r.$$

This scalar appears repeatedly, so write

$$g_{\eta,r} = \frac{y_r - p_r}{p_r(1 - p_r)} s_r.$$

Then the person-level gradient is

$$\nabla_{\theta_j} \ell_j(\theta_j) = -Q_{0j}(\theta_j - \mu_j) + \sum_{r \in \mathcal{R}_j} g_{\eta,r} a_r.$$

Laplace precision

The current code uses an expected-information-style precision matrix

$$H_j = Q_{0j} + \sum_{r \in \mathcal{R}_j} w_r a_r a_r^\top,$$

with

$$w_r = \frac{s_r^2}{p_r(1 - p_r)}.$$

This is positive semidefinite by construction and is then stabilized with diagonal jitter before Cholesky factorization.

Numerical update

For each person, the C++ routine uses damped Newton updates:

$$\theta_j^{\text{new}} = \theta_j + \lambda H_j^{-1} \nabla_{\theta_j} \ell_j(\theta_j),$$

with backtracking over $\lambda \in \{1, \frac{1}{2}, \frac{1}{4}, \dots\}$ until the person log posterior increases.

The outputs of this step are:

- posterior mode $\hat{\theta}_j$
- posterior precision H_j
- Cholesky factor of H_j
- $\log |H_j|$
- optionally H_j^{-1}

Item-step surrogate objective

After the person step, the current `laplace_em` path freezes the person modes $\hat{\theta}_j$, but not the Laplace precision itself. During the item step, the precision is recomputed from the current row-effective item parameters and the frozen modes. The surrogate is therefore

$$\tilde{L}(\xi) = \sum_{j=1}^N \left[\sum_{r \in \mathcal{R}_j} \log p(y_r \mid \hat{\theta}_j, \xi) + \frac{K}{2} \log(2\pi) - \frac{1}{2} \log |H_j(\xi \mid \hat{\theta}_j)| \right] + \log \pi(\xi),$$

where ξ denotes the item/global parameters being updated.

where the item-step precision is

$$H_j(\xi \mid \hat{\theta}_j) = Q_{0j} + \sum_{r \in \mathcal{R}_j} w_r(\xi, \hat{\theta}_j) a_r(\xi) a_r(\xi)^\top.$$

Important implementation details:

- the person modes are treated as fixed during the item step
- the Laplace precision term is recomputed and differentiated with respect to the current item/global parameters, conditional on those fixed person modes
- the prior on θ_j is constant with respect to ξ once $\hat{\theta}_j$ and Q_{0j} are frozen, so it does not enter the item-step objective

This is therefore a generalized-EM surrogate, not the exact Laplace marginal likelihood with full differentiation through $\hat{\theta}_j(\xi)$.

Implemented rowwise Laplace gradient

For a single row r , let

$$\Sigma_j = H_j^{-1}, \quad A_r = a_r^\top \Sigma_j a_r.$$

The item-step C++ backend differentiates the frozen-mode surrogate including the $-\frac{1}{2} \log |H_j|$ term.

The implemented rowwise gradients are:

$$\frac{\partial \tilde{L}_r}{\partial a_r} = g_{\eta, r} z_r - \frac{1}{2} \left(\frac{\partial w_r}{\partial \eta_r} z_r A_r + 2 w_r \Sigma_j a_r \right),$$

where z_r is the effective row ability vector used in the row predictor.

Componentwise, for loading component k ,

$$\frac{\partial \tilde{L}_r}{\partial a_{rk}} = g_{\eta,r} z_{rk} - \frac{1}{2} \left(\frac{\partial w_r}{\partial \eta_r} z_{rk} A_r + 2w_r (\Sigma_j a_r)_k \right).$$

Similarly,

$$\frac{\partial \tilde{L}_r}{\partial b_r} = -g_{\eta,r} + \frac{1}{2} \frac{\partial w_r}{\partial \eta_r} A_r,$$

$$\frac{\partial \tilde{L}_r}{\partial c_r} = \frac{\partial \log p(y_r \mid \hat{\theta}_j, \xi)}{\partial c_r} - \frac{1}{2} \frac{\partial w_r}{\partial c_r} A_r,$$

$$\frac{\partial \tilde{L}_r}{\partial d_r} = \frac{\partial \log p(y_r \mid \hat{\theta}_j, \xi)}{\partial d_r} - \frac{1}{2} \frac{\partial w_r}{\partial d_r} A_r.$$

The C++ backend computes these row-level derivatives on the transformed scale. The R layer then maps them back to the raw optimization parameters by chain rule and accumulates them onto:

- free loadings
- item intercepts
- guessing / upper-asymptote raw parameters
- item covariate coefficients
- item prior means, when free

Item priors

The current implementation adds Gaussian priors on raw item/global parameters in the R layer:

$$\alpha \sim N(\mu_\alpha, \sigma_\alpha^2), \quad b \sim N(\mu_b, \sigma_b^2), \quad \gamma \sim N(\mu_c, \sigma_c^2), \quad \delta \sim N(\mu_d, \sigma_d^2),$$

with analogous zero-centered Gaussian priors on item covariate coefficients when enabled.

These priors contribute both to the objective and to the gradient.

Outer loop

The current `laplace_em` implementation proceeds as follows.

1. Initialization

Run the existing Stan-based JML/MAP fit to obtain a valid initial state for:

- item parameters
- person abilities
- item and person regression coefficients

2. Working prior anchoring

Construct a fixed working latent prior from the initialization fit:

- latent means from the initialized abilities
- latent standard deviations from the initialized abilities
- latent correlation from the initialized abilities

This anchored covariance is inverted to obtain the working prior precision Q_{0j} . The purpose is to keep the Laplace loop on the same latent scale as the initialization fit, rather than reverting to a very broad default prior.

3. Outer iteration

For outer iteration $t = 1, \dots, T$:

1. Person step: update $\hat{\theta}_j^{(t)}$ and $H_j^{(t)}$
2. Item step: optimize the frozen-mode surrogate over ξ , recomputing $H_j(\xi \mid \hat{\theta}_j^{(t)})$ inside the item objective
3. Person refresh: recompute $\hat{\theta}_j^{(t)}$ and $H_j^{(t)}$ under the updated ξ
4. Optional latent rescaling: if the model structure allows it, rescale the latent state back toward the anchored mean and standard deviation, then refresh the person step again
5. Evaluate the surrogate objective and step sizes

4. Stopping rule

The current code stops when, on two consecutive outer iterations,

- relative objective improvement is below `laplaceTol`
- item-parameter RMS movement is below `laplaceTol`
- person-mode RMS movement is below `laplaceTol`

otherwise it stops at `laplaceOuterIter`.

What this algorithm is, and is not

The current implementation is best described as:

- a Laplace-based generalized EM method
- with exact rowwise gradients for the frozen-mode surrogate
- but not exact differentiation through $\hat{\theta}_j(\xi)$

So it is not yet full exact Laplace marginal maximum likelihood.

More specifically:

- The person step uses a Laplace approximation based on the conditional posterior mode and expected-information precision.
- The item step includes the Laplace correction term $-\frac{1}{2} \log |H_j|$ and differentiates it with respect to item/global parameters, but treats $\hat{\theta}_j$ as fixed within each item optimization pass.
- The latent prior precision used in the Laplace loop is fixed after initialization.

Current implementation limitations

The current code has several important limitations.

1. `Abilitybeta` is not updated inside `laplace_em` when person predictors are present.
2. The person precision uses the expected-information form $w_r a_r a_r^\top$, not the full observed Hessian of the conditional posterior.

3. The item step is a frozen-mode surrogate step, not full exact differentiation through the mode map $\hat{\theta}_j(\xi)$.
4. Only training rows contribute to the fitting objective.
5. The final reported item parameters are reconstructed using mean item-covariate values, and final reported person abilities are reconstructed using mean person-covariate values.

Mapping to code

The main pieces live in:

- `R/laplaceBackend.R`
- `src/laplace_backend.cpp`
- `R/fitIRT.R`

Conceptually:

- `bigIRT_laplace_row_effective()` constructs rowwise effective parameters
- `bigIRT_laplace_person_step()` runs the person-mode / precision update
- `bigIRT_laplace_item_objective()` and `bigIRT_laplace_optimize_item()` evaluate and optimize the frozen-mode Laplace surrogate
- the `laplace_em` block in `fitIRT()` runs the outer generalized-EM loop